

Graph on 3 vertices with 1 edges.

Adjacencies:

```
3 :
2 : 0
0 : 2
```

In the above examples, if we do not intend to modify the original graph, we can make a copy of it using `copy_graph()`, and then operate on the copy, as follows:

```
(%i8) h: copy_graph(g);
(%o8)          GRAPH(3 vertices, 1 edges)
(%i9) add_vertex(1, h)$
(%i10) print_graph(h)$
```

Graph on 4 vertices with 1 edges.

Adjacencies:

```
1 :
0 : 2
2 : 0
3 :
```

```
(%i11) print_graph(g)$ /* Notice g is unmodified */
```

Graph on 3 vertices with 1 edges.

Adjacencies:

```
3 :
2 : 0
0 : 2
```

```
(%i12) quit();
```

Advanced graph creations

New graphs can also be created based on existing graphs and their properties by various interesting operations. A few of them are listed below:

- `underlying_graph(<dg>)` - Returns the underlying graph of the directed graph <dg>
- `complement_graph(<g>)` - Returns the complement graph of graph <g>
- `line_graph(<g>)` - Returns a graph that represents the adjacencies between the edges of graph <g>
- `graph_union(<g1>, <g2>)` - Returns a graph with edges and vertices of both graphs <g1> and <g2>
- `graph_product(<g1>, <g2>)` - Returns the Cartesian product of graphs <g1> and <g2>

Here are some examples to demonstrate the simpler functions:

```
$ maxima -q
(%i1) load(graphs)$
...
0 errors, 0 warnings
(%i2) g: create_graph(4, [[0, 1], [0, 2], [0, 3]], directed =
true);
(%o2)          DIGRAPH(4 vertices, 3 arcs)
(%i3) print_graph(g)$
```

Digraph on 4 vertices with 3 arcs.

Adjacencies:

```
3 :
2 :
1 :
0 : 3 2 1
```

```
(%i4) h: underlying_graph(g);
```

```
(%o4)          GRAPH(4 vertices, 3 edges)
```

```
(%i5) print_graph(h)$
```

Graph on 4 vertices with 3 edges.

Adjacencies:

```
0 : 1 2 3
1 : 0
2 : 0
3 : 0
```

```
(%i6) print_graph(complement_graph(h))$
```

Graph on 4 vertices with 3 edges.

Adjacencies:

```
3 : 2 1
2 : 3 1
1 : 3 2
0 :
```

```
(%i7) print_graph(graph_union(h, complement_graph(h)))$
```

Graph on 8 vertices with 6 edges.

Adjacencies:

```
4 :
5 : 6 7
6 : 5 7
7 : 5 6
3 : 0
2 : 0
1 : 0
0 : 3 2 1
```

```
(%i8) quit();
```

Basic graph properties

`graph_order(<g>)`, `vertices(<g>)` returns the number of vertices and the list of vertices, respectively, in the graph <g>. `graph_size(<g>)`, `edges(<g>)` returns the number of edges and the list of edges, respectively, in the graph <g>.

A graph is a collection of vertices and edges. Hence, most of its properties are centred around them. The following are graph related predicates provided by the graphs package of Maxima:

- `is_graph(<g>)` - returns 'true' if <g> is a graph, and 'false' otherwise
- `is_digraph(<g>)` - returns 'true' if <g> is a directed graph, and 'false' otherwise
- `is_graph_or_digraph(<g>)` - returns 'true' if <g> is a graph or a directed graph, and 'false' otherwise
- `is_connected(<g>)` - returns 'true' if graph <g> is